

Python: Advanced Cheat Sheet

Type Hints

Annotations document expected types. They are **not enforced at runtime** — they exist for editors, linters, and type checkers like mypy.

```
def greet(name: str, times: int = 1) -> str:
    return f"hi {name} " * times

age: int = 30                                # variable annotation

# Built-in generics (Python 3.9+)
nums: list[int] = [1, 2, 3]
scores: dict[str, int] = {"a": 1}
pair: tuple[int, str] = (1, "x")

# "Optional" / union types (Python 3.10+ uses the | operator)
def find(x: int) -> str | None:              # returns a str OR None
    return None

from typing import Optional, Callable, Any
val: Optional[int] = None                    # same as: int | None
cb: Callable[[int], str] = str               # a function taking int, returning str
anything: Any = "escape hatch"              # disables type checking for this name

# Final: declares a constant — enforced by TYPE CHECKERS only; the
# interpreter never blocks the reassignment (Python has no true const)
from typing import Final
MAX_SIZE: Final = 100
MAX_SIZE = 200                             # runs fine; mypy reports an error
# For runtime-protected constants use an Enum: members can't be
# reassigned (Color.RED = 99 -> AttributeError) — see Enums below.
```

Private Attributes & Name Mangling

Python has **no truly private** members — privacy is by convention:

- `__name` — a hint meaning "internal, please don't touch". Nothing enforces it.
- `__name` — triggers *name mangling*: Python rewrites it to `__ClassName__name`, mainly to avoid accidental clashes in subclasses (not real hiding).

```
class Account:
    def __init__(self):
        self.owner = "Alice"                # public
        self.__balance = 100                # "internal" by convention
        self.__pin = 1234                    # mangled to __Account__pin

a = Account()
a.owner                # "Alice"
a.__balance             # 100 -> still accessible; underscore is just a signal
# a.__pin               # AttributeError
a.__Account__pin        # 1234 -> the real (mangled) name
```

Properties (@property)

The Pythonic way to add getters/setters/validation while keeping plain attribute syntax (no parentheses on access).

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):          # getter: read like an attribute
        return self._radius

    @radius.setter
    def radius(self, value):   # setter: runs on assignment
        if value < 0:
            raise ValueError("radius must be >= 0")
        self._radius = value

    @property
    def area(self):            # computed, read-only (no setter defined)
        return 3.14159 * self._radius ** 2

c = Circle(5)
c.radius          # 5          -> calls the getter (no parentheses!)
c.radius = 10     # calls the setter (validates first)
c.area            # 314.159    -> recomputed each access
# c.area = 1      # AttributeError: can't set (read-only)
```

Dynamic Attributes

Attributes can be added or removed on instances (and classes) at runtime — they live in a `__dict__`.

```
class Dog:
    pass

d = Dog()
d.name = "Rex"          # add an attribute at runtime
d.name                  # "Rex"
del d.name               # remove it
d.__dict__               # {} -> instance attributes are stored here

# Adding to the CLASS affects all instances — even existing ones
existing = Dog()
Dog.species = "canine"   # add a class attribute at runtime
existing.species          # "canine" -> the already-created instance sees it
Dog().species            # "canine" -> and so does a brand-new one
del Dog.species          # remove it again (gone for all instances)

# Methods are just CALLABLE attributes — one can be added at runtime
Dog.speak = lambda self: "woof"
d.speak()                # "woof" -> existing instances get it too

# Functions are objects too, and DO accept arbitrary attributes
def greet():
    return "hi"

greet.calls = 0          # e.g. tag metadata onto the function
greet.__dict__           # {'calls': 0}
```

```

# Built-in ("primitive") types are sealed: neither their instances nor
# the types themselves accept new attributes or methods
# (5).x = 1          # AttributeError: ...no __dict__ for setting attributes
# int.x = 1          # TypeError: cannot set 'x' attribute of
#                   #             immutable type 'int'
# str.shout = f       # TypeError - no monkey-patching built-ins

# The way to "extend" a built-in is to subclass it
class MyStr(str):
    def shout(self):
        return self.upper() + "!"

MyStr("hi").shout()    # "HI!"

# __slots__ PREVENTS arbitrary attributes (less memory, fixed names)
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Point(1, 2)
# p.z = 3            # AttributeError: 'Point' object has no attribute 'z'
# Point objects also have no __dict__

```

Introspection

Inspect and manipulate attributes by name at runtime.

```

class User:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"hi {self.name}"

u = User("Alice")

hasattr(u, "name")      # True
getattr(u, "name")      # "Alice"
getattr(u, "age", 0)    # 0    -> default returned if attribute is missing
setattr(u, "age", 30)   # same as: u.age = 30
delattr(u, "age")       # same as: del u.age

# Enumerate attributes & methods
dir(u)                  # list ALL names (attributes + methods + inherited)
vars(u)                 # {'name': 'Alice'} -> instance attrs (its __dict__)
u.__dict__              # same as vars(u)
User.__dict__           # the class's own members (methods live here)

# Only the "own" (non-dunder) names:
[a for a in dir(u) if not a.startswith("__")]    # ['greet', 'name']

# Just the callable attributes (i.e. the methods):
[a for a in dir(u) if callable(getattr(u, a))
 and not a.startswith("__")]                    # ['greet']

# Type checks
isinstance(u, User)          # True
isinstance(5, (int, float))  # True -> tuple means "any of these types"
issubclass(bool, int)        # True -> bool is a subclass of int

```

```
type(u) is User          # True -> exact type, ignores inheritance
```

Abstract Base Classes (Interfaces)

Python has no `interface` keyword, but `abc` provides the equivalent: a base class that defines required methods. Subclasses **must** implement every `@abstractmethod`, or they can't be instantiated.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        ...

    def describe(self):
        return f"area = {self.area()}"

# shape = Shape()          # TypeError: can't instantiate abstract class

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

sq = Square(4)
sq.area()          # 16
sq.describe()      # "area = 16"
```

Often you don't even need this — thanks to duck typing, any object with the right methods works. ABCs are for when you want to *enforce* that contract.

Iterators (Full Protocol)

An **iterator** implements `__iter__` (returns the iterator) and `__next__` (returns the next value, or raises `StopIteration` when done). This is what `for` loops use under the hood. (Generators are the shortcut for this.)

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

for n in Countdown(3):
    print(n)

# Splitting iterable from iterator lets you loop multiple times.
# Here __iter__ returns a FRESH iterator on each call:
class Numbers:
    def __init__(self, data):
```

```

        self.data = data
    def __iter__(self):
        return iter(self.data)          # delegate to the list's own iterator

nums = Numbers([1, 2, 3])
list(nums)          # [1, 2, 3]
list(nums)          # [1, 2, 3] again -> reusable

```

Dataclasses

Auto-generate `__init__`, `__repr__`, `__eq__`, etc. from typed fields — great for plain data containers.

```

from dataclasses import dataclass, field

@dataclass
class Point:
    x: int
    y: int = 0                # default value
    tags: list = field(default_factory=list)  # mutable default -> factory

p = Point(1, 2)
p                # Point(x=1, y=2, tags=[]) -> generated __repr__
p == Point(1, 2) # True          -> generated __eq__
p.x              # 1

# frozen=True makes instances immutable (also hashable)
@dataclass(frozen=True)
class Config:
    name: str
    debug: bool = False

cfg = Config("prod")
# cfg.debug = True    # FrozenInstanceError

```

Enums

A fixed set of named constant values — clearer and safer than bare strings or magic numbers.

```

from enum import Enum

class Color(Enum):
    RED = 1
    GREEN = 2
    BLUE = 3

Color.RED        # <Color.RED: 1>
Color.RED.name   # "RED"
Color.RED.value   # 1
Color(2)         # <Color.GREEN: 2> -> look up a member by value
Color["RED"]     # <Color.RED: 1>   -> look up by name
list(Color)      # [<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 3>]

Color.RED == Color.RED    # True    -> compare by identity
Color.RED == 1            # False   -> a member is NOT its raw value

# auto() assigns the values for you
from enum import auto

```

```
class Status(Enum):
    PENDING = auto()      # 1
    ACTIVE = auto()        # 2
    DONE = auto()          # 3
```

`__repr__` vs `__str__` (and Their Absence)

Two ways an object turns into text:

- `__repr__` — for **developers**: unambiguous, ideally something that could recreate the object. Used by the REPL, the debugger, and when an object appears **inside a container** (e.g. printing a list of objects).
- `__str__` — for **end users**: readable/friendly. Used by `print()`, `str()`, and f-strings.

```
# 1. NEITHER defined -> ugly default from object
class Bare:
    def __init__(self, x):
        self.x = x

str(Bare(1))          # "<__main__.Bare object at 0x...>" (unhelpful)

# 2. ONLY __repr__ defined -> str() FALLS BACK to it
class ReprOnly:
    def __init__(self, x):
        self.x = x
    def __repr__(self):
        return f"ReprOnly(x={self.x})"

r = ReprOnly(1)
repr(r)               # "ReprOnly(x=1)"
str(r)                # "ReprOnly(x=1)" -> falls back to __repr__
print(r)              # ReprOnly(x=1)
print([r])            # [ReprOnly(x=1)] -> containers always use __repr__

# 3. ONLY __str__ defined -> repr() stays the UGLY default (no fallback!)
class StrOnly:
    def __str__(self):
        return "friendly"

s = StrOnly()
str(s)                # "friendly"
repr(s)               # "<__main__.StrOnly object at 0x...>" -> still ugly
# print([s])          # uses repr -> shows the ugly form, NOT "friendly"

# 4. BOTH defined -> each context picks the right one
class Temp:
    def __init__(self, c):
        self.c = c
    def __repr__(self):
        return f"Temp({self.c})"      # for developers
    def __str__(self):
        return f"{self.c}°C"          # for users

t = Temp(20)
str(t)                # "20°C"
repr(t)               # "Temp(20)"
```

Rule of thumb: if you only define one, define `__repr__` — `str()` falls back to it, but never the reverse.

Threading & the GIL

```
import threading

def worker(n):
    print(f"working on {n}")

t = threading.Thread(target=worker, args=(1,))
t.start()
t.join()          # wait for the thread to finish

# A Lock protects shared state from concurrent access
lock = threading.Lock()
with lock:
    # only one thread inside at a time
    pass
```

Key caveat: CPython has a Global Interpreter Lock (GIL), so threads do **not** run Python bytecode in parallel. Threading only speeds up **I/O-bound** work (network, disk, waiting), where threads sit idle. For **CPU-bound** work, use multiprocessing instead.

Multiprocessing

Separate processes sidestep the GIL and give **true parallelism** — best for CPU-bound work. Each process has its own memory.

```
from multiprocessing import Pool

def square(n):
    return n * n

# The __main__ guard is REQUIRED for multiprocessing to work safely
if __name__ == "__main__":
    with Pool(4) as pool:
        results = pool.map(square, [1, 2, 3, 4])    # [1, 4, 9, 16]
        print(results)
```

Async / Await

```
import asyncio

async def fetch(n):
    # 'async def' creates a coroutine
    await asyncio.sleep(0.1)    # non-blocking pause (yields control)
    return n * 2

async def main():
    # gather schedules coroutines concurrently and waits for all of them
    results = await asyncio.gather(fetch(1), fetch(2), fetch(3))
    return results              # [2, 4, 6]

asyncio.run(main())            # starts the event loop and runs main()
```

Key idea: async is *concurrency*, *not parallelism* — a single thread that cooperatively switches tasks at each `await`. Ideal for handling many I/O waits at once (e.g. hundreds of network requests), not for CPU-bound work.

Paths with `pathlib`

The modern, object-oriented way to handle file paths (preferred over string manipulation and most of the older `os.path`).

```
from pathlib import Path

p = Path("folder") / "sub" / "file.txt"  # build paths with the / operator
p.name          # "file.txt"
p.stem          # "file"              name without the suffix
p.suffix        # ".txt"              the extension
p.parent        # Path("folder/sub")
p.parts         # ('folder', 'sub', 'file.txt')

Path.cwd()      # current working directory
Path.home()     # user's home directory
p.with_suffix(".md")  # Path("folder/sub/file.md")

# Filesystem operations (shown commented; they touch real files):
# p.exists() / p.is_file() / p.is_dir()
# p.read_text() / p.write_text("hi")
# Path("data").mkdir(exist_ok=True)      # create a directory
# for child in Path(".").iterdir(): ...   # list a directory
# for py in Path(".").glob("*.py"): ...   # match a pattern
```